Accessibility Audit Report — Logfire Python SDK

Date: 2025-07-14
Auditor: Accessibility Audit Agent (SDL-aligned)
Project: Logfire Python SDK v4.31.0
Scope: CLI tool, console exporter output, MkDocs documentation (`docs/`)
Standard: WCAG 2.1 / 2.2 (Levels A, AA, AAA)
SDL Framework: Microsoft Security Development Lifecycle — accessibility issues classified as Implementation-phase or Design-phase defects

---

Scope Assessment by Component

Component	Accessibility Applicable?	Rationale
Python SDK API	❌ Not applicable	No interactive UI; developer-facing Python library
CLI tool (`logfire` command)	☑ **In scope**	Interactive terminal UI with user-facing output
Console exporter (Rich output)	☑ **In scope**	Terminal output consumed by humans and piped to tools
MkDocs documentation (`docs/`)	☑ **In scope**	Public-facing HTML website
OTLP/Protobuf exporter	❌ Not applicable	Machine-to-machine protocol, no human-readable UI
Test utilities	❌ Not applicable	Developer-facing testing helpers

WCAG Conformance Summary

Level	Status	Notes
WCAG 2.1 Level A	❌ **Non-conformant**	3 Level A failures identified (color, role, alt text)
WCAG 2.1 Level AA	❌ **Non-conformant**	Color contrast failure in docs light theme
WCAG 2.1 Level AAA	❌ **Non-conformant**	Animation from interactions not suppressed under prefers-reduced-motion

Quick Wins (Highest Impact, Lowest Effort)

1. **Add `[ERROR]` / `[WARN]` text prefix to console exporter** — single-line change in `_span_text_parts()` eliminates the only Level A color-only failure in the core SDK.
2. **Remove `role="presentation"` from search results list** — one-word deletion in `docs/overrides/partials/search.html`; restores screen-reader list semantics immediately.
3. **Add `@media (prefers-reduced-motion: reduce)` block to tweaks.css** — two-line CSS addition disables the login button hover animation.
4. **Document `NO_COLOR` env var support** — zero code change; add a note to CLI reference docs and `ConsoleOptions` docstring.
5. **Add descriptive alt text to high-traffic screenshots** — fix `docs/index.md`, `docs/concepts.md`, and `docs/why.md` screenshots first; highest user visibility with low effort.

Findings

[CRITICAL] — Log Severity Conveyed by Color Alone

- **SDL Phase**: Implementation
- **WCAG Criterion**: 1.4.1 Use of Color (Level A)
- **Category**: Console Exporter
- **File**: `logfire/_internal/exporters/console.py`, lines 207–216

Description

In `_span_text_parts()`, the log severity level is communicated *exclusively* through terminal text color:

```
if level >= _ERROR_LEVEL:
    parts += [(msg, 'red')]        # error
elif level >= _WARN_LEVEL:
    parts += [(msg, 'yellow')]     # warning
else:
    parts += [(msg, '')]           # info — no color distinction
```

No text label (e.g., `[ERROR]`, `[WARN]`, `[INFO]`) is ever prepended to or appended after the message. When colors are disabled — by piping output to a file, setting `LOGFIRE_CONSOLE_COLORS=never`, or running in a non-TTY environment — *all* severity levels are visually identical. Red–green color blindness (affecting approximately 8% of men) also makes it impossible to distinguish `ERROR` (red) from default text.

Impact

- Color-blind users cannot distinguish warnings from errors.
- Users piping `logfire` output to log-aggregation tools, CI log files, or monitoring systems see all messages without severity markers.
- Operators on call who use high-contrast terminal themes may miss critical errors.
- Any accessibility tooling that captures terminal output (e.g., BRLTTY, screen-reader-driven terminal emulators) provides no severity information to the user.

Recommendation

Prepend a text-based severity tag to every message. In `_span_text_parts()`:

```
LEVEL_LABELS = {
    # level_num_threshold : label
    _ERROR_LEVEL: 'ERROR',
    _WARN_LEVEL:  'WARN ',
}

def _get_level_label(level: int) -> str:
    if level >= _ERROR_LEVEL:
        return 'ERROR'
    elif level >= _WARN_LEVEL:
        return 'WARN '
    else:
        return 'INFO '

# Then in _span_text_parts():
label = _get_level_label(level)
parts += [(f'[{label}] ', 'red' if level >= _ERROR_LEVEL else 'yellow' if level >= _WARN_LEVEL else 'dim')]
parts += [(msg, 'red' if level >= _ERROR_LEVEL else '')]
```

This change preserves the existing colored output for sighted users while adding machine-readable and color-independent severity markers.

[HIGH] — Primary Brand Color (#E520E9) Fails WCAG AA Contrast on Light Background

- **SDL Phase**: Design
- **WCAG Criterion**: 1.4.3 Contrast (Minimum) (Level AA)
- **Category**: Documentation Color Contrast
- **File**: `docs/extra/tweaks.css`, lines 2–6

Description

The custom CSS sets the Material for MkDocs primary color to:

```
:root {
  --md-primary-fg-color: #E520E9;
  --md-primary-fg-color--light: #E520E9;
  --md-primary-fg-color--dark: #E520E9;
}
```

Contrast ratio analysis:

Foreground	Background	Ratio	WCAG AA Normal Text (4.5:1)	WCAG AA Large Text / UI (3:1)
`#E520E9`	`#FFFFFF` (light mode)	**3.67:1**	❌ FAIL	☑ PASS
`#E520E9`	`#1A0312` (dark mode)	**5.32:1**	☑ PASS	☑ PASS
`#E520E9`	`#321A2C` (nav header)	**3.88:1**	❌ FAIL	☑ PASS

Material for MkDocs uses the primary color (`--md-primary-fg-color`) for:

- Navigation link text in the left sidebar (normal text weight, ~14px)
- Inline text links within page body content
- Active tab indicator text
- Search highlight color (via `color: var(--md-primary-fg-color)` in tweaks.css, line 91)
- Focus ring on interactive elements

In the **light theme** (default), links and highlighted navigation items using this color on white fail WCAG AA minimum contrast for normal-sized text.

Impact

Users with low vision, poor-quality monitors, or bright ambient lighting will experience reduced readability for all hyperlinks and

navigation items in the default light theme. The search results page also highlights matched text using this failing color.

Recommendation

Define a separate light-mode variant of the primary color that achieves ≥ 4.5:1 against white. For example, `#9900A0` achieves approximately 5.1:1 against white while maintaining the brand hue. Use a CSS media query or scheme-specific override:

```
/* Light mode: use a darker shade for sufficient contrast */
[data-md-color-scheme="default"] {
  --md-primary-fg-color: #9900A0;
  --md-primary-fg-color--light: #9900A0;
}

/* Dark mode: current color is fine */
[data-md-color-scheme="slate"] {
  --md-primary-fg-color: #E520E9; /* 5.32:1 on #1A0312 — PASS */
}
```

[HIGH] — Search Results List Has role="presentation", Removing Screen-Reader Semantics

- **SDL Phase**: Implementation
- **WCAG Criterion**: 4.1.2 Name, Role, Value (Level A)
- **Category**: Documentation ARIA / Search
- **File**: `docs/overrides/partials/search.html`, line 27

Description

The ordered list that contains Algolia search results is declared with `role="presentation"`:

```
<ol class="md-search-result__list" id="hits" role="presentation"></ol>
```

`role="presentation"` (equivalent to `role="none"`) explicitly strips all native list semantics from this element and its children. As a result:

- Screen readers will not announce the number of search results.
- Individual results will not be identified as list items.
- Navigation by list (a common screen-reader shortcut) will not function.
- `aria-live` region updates (if any) will not provide a result count.

Impact

VoiceOver, NVDA, and JAWS users will experience search results as an unmarked sequence of links with no structural grouping. The absence of a result count (e.g., "8 results") means users cannot assess whether their search was successful without manually reading every result.

Recommendation

Remove `role="presentation"` from the `<ol>`. If the intent was to prevent double-announcement of the list structure, use `aria-label` instead to give it a meaningful name:

```
<ol class="md-search-result__list" id="hits" aria-label="Search results" aria-live="polite" aria-atomic="false"></ol>
```

Adding `aria-live="polite"` on the result container (or a wrapping `<div>`) will also announce when new results appear without being disruptive.

[MEDIUM] — Unicode-Only Status Indicators in CLI Instrumentation Summary

- **SDL Phase**: Implementation
- **WCAG Criterion**: 1.1.1 Non-text Content (Level A — applied by analogy to terminal output)
- **Category**: CLI Terminal Output
- **File**: `logfire/_internal/cli/run.py`, lines 227–231

Description

The `instrumented_packages_text()` function builds a checklist using Unicode symbols as the primary (and sole) visual status indicator:

```
text.append(f'✓ {base_pkg} (installed and instrumented)\n', style='green')
# ...
text.append(f'⚠ {base_pkg} (installed but not automatically instrumented)\n', style='yellow')
```

And in `get_recommendation_texts()`:

```
recommended_text.append(f'☐ {instrumented_pkg} (need to install {pkg_name})\n', style='grey50')
```

Screen readers in terminal emulators may announce these symbols by their Unicode names:

- `✓` (U+2713) → "CHECK MARK" — acceptable
- `⚠` (U+26A0 + variation selector) → "WARNING SIGN" or "⚠" — inconsistent across AT
- `☐` (U+2610) → "BALLOT BOX" — not intuitive for "package not installed"

Additionally, these symbols rely on color (`green`, `yellow`, `grey50`) as a secondary differentiator, compounding the color-only issue.

Impact

Users running `logfire inspect` or `logfire run` via a screen reader (e.g., via SSH in a screen-reader-accessible terminal session) may hear confusing or meaningless Unicode names. The `☐ ballot box` in particular does not convey "missing instrumentation package" without surrounding context.

Recommendation

Replace or supplement Unicode symbols with unambiguous text tags:

```
text.append(f'[OK]      {base_pkg} (installed and instrumented)\n', style='green')
text.append(f'[WARNING] {base_pkg} (installed but not automatically instrumented)\n', style='yellow')
recommended_text.append(f'[MISSING] {pkg_name} — needed to instrument {instrumented_pkg}\n', style='grey50')
```

If preserving the Unicode symbols for visual appeal, add a text legend at the top of the panel:

```
✓ = installed and instrumented  ⚠ = not auto-instrumented  ☐ = not installed
```

[MEDIUM] — Search Result Links Have tabindex="-1", Blocking Tab Navigation

- **SDL Phase**: Implementation
- **WCAG Criterion**: 2.1.1 Keyboard (Level A)
- **Category**: Documentation Keyboard Navigation
- **File**: `docs/javascripts/algolia-search.js`, line 95

Description

The Algolia Hits widget template renders every search result link with `tabindex="-1"`:

```
return html`
  <a href="${hit.abs_url}" class="md-search-result__link" tabindex="-1">
    ...
  </a>`
```

`tabindex="-1"` removes these links from the natural Tab order. While the JavaScript in the same file implements ArrowDown/ArrowUp keyboard navigation (lines 51–67), this navigation mode is:

1. Non-discoverable — there is no visible indication that arrow keys should be used.
2. Non-standard — users expect Tab to move between interactive elements.
3. Incomplete — no Escape key handler is registered to return focus to the search box from within results.

Impact

Keyboard-only users who Tab into the search field will be unable to Tab further to navigate search results. They are expected to use ArrowDown (which is documented nowhere in the UI) and will likely assume keyboard navigation of search results is impossible.

Recommendation

Option A (preferred): Change `tabindex="-1"` to `tabindex="0"` and rely on the existing arrow-key handlers as *supplemental* navigation, while restoring Tab as the primary method.

Option B: Keep `tabindex="-1"` but add a visible hint when the search is active:

```
<div class="md-search__keyboard-hint" aria-live="polite">
  Use ↑↓ arrow keys to navigate results, Enter to select
</div>
```

Also add an `Escape` key handler to return focus to the search input:

```
document.querySelector('#hits').addEventListener('keydown', (event) => {
  if (event.key === 'Escape') {
    document.querySelector('#searchbox').focus();
    event.preventDefault();
  }
  // ... existing ArrowDown/ArrowUp handlers
});
```

[MEDIUM] — Screenshot Images Have Non-Descriptive Alt Text Throughout Documentation

- **SDL Phase**: Implementation
- **WCAG Criterion**: 1.1.1 Non-text Content (Level A)
- **Category**: Documentation Images
- **Files**: Multiple (representative examples below)

Description

Screenshots throughout the documentation use minimal alt text that names the product or integration but does not describe the UI content of the image:

File	Current Alt Text	Problem
`docs/integrations/llms/openai.md:40`	`Logfire OpenAI`	Doesn't describe span tree, token counts, or conversation flow shown

File	Current Alt Text	Problem
`docs/integrations/llms/openai.md:45`	`Logfire OpenAI Arguments`	Doesn't describe argument values or response data shown
`docs/why.md:67`	`Logfire FastAPI screenshot`	Doesn't describe request parameters, HTTP method, or trace depth
`docs/why.md:77`	`Logfire hello world screenshot`	Doesn't describe output values, age calculation, or log structure
`docs/concepts.md:27`	`Spans`	Single-word alt text for a complex UI screenshot
`docs/concepts.md:38`	`Trace`	Single-word alt text
`docs/guides/onboarding-checklist/integrate.md:21`	`Logfire inspect command`	Doesn't describe which packages are listed or what action to take

These screenshots convey important contextual information — the actual output that users should expect — and the current alt text fails to communicate this to screen reader users.

Impact
Screen reader users cannot understand what is shown in screenshots that are central to understanding how Logfire works. For tutorial and onboarding pages, the screenshots often show the expected output; without descriptive alt text, visually impaired users cannot follow the tutorial effectively.

Recommendation
Write descriptive alt text that describes the content visible in the screenshot. For screenshots that are *functionally* redundant to code examples above them, use `alt=""` to mark them as decorative. Examples:

```
<!-- Descriptive for informational screenshots: -->
![Logfire Live View showing an OpenAI span with GPT-4 conversation messages,
response text, and token usage breakdown (prompt: 42 tokens, completion: 87 tokens)](...)

<!-- Decorative when the screenshot supplements inline code: -->
<figure markdown="span">
  ![](../../images/logfire-screenshot-openai.png){ width="500" alt="" }
  <figcaption>OpenAI span and conversation in Logfire Live View</figcaption>
</figure>
```

[MEDIUM] — NO_COLOR Environment Variable Not Explicitly Documented or Tested

- **SDL Phase**: Implementation
- **WCAG Criterion**: 1.4.1 Use of Color (Level A — principle)
- **Category**: CLI / Console Output Accessibility
- **File**: `logfire/_internal/config_params.py` (CONSOLE_COLORS param); `docs/reference/cli.md`

Description

The console exporter provides `LOGFIRE_CONSOLE_COLORS=never` as the mechanism to disable color output, but does not document or explicitly test support for the widely-adopted `NO_COLOR` environment variable standard (https://no-color.org/). Rich (the underlying library) does check for `NO_COLOR` internally, so the behavior works incidentally, but:

1. It is not documented anywhere in Logfire's public API or CLI reference.
2. It is not explicitly tested — if Rich's behavior changes, Logfire would silently break.
3. Users who rely on `NO_COLOR` for accessibility cannot verify this works from Logfire's docs.

`NO_COLOR` is used by CI systems, accessibility-driven terminals, and users with color vision deficiencies to globally disable all color output across all compatible CLI tools.

Impact

Users who have set `NO_COLOR=1` in their environment (e.g., in `.bashrc`, as an OS-level accessibility setting) may be uncertain whether Logfire respects it. Operators who configure `NO_COLOR` centrally in CI environments may get unexpected colored output if Rich's behavior changes.

Recommendation

1. Add explicit documentation: in `docs/reference/cli.md` and the `ConsoleOptions` docstring, note that `NO_COLOR=1` disables all ANSI color output (via Rich's built-in support).
2. Add a test case: `test_no_color_env_var()` that sets `NO_COLOR=1` and verifies the console exporter produces plain-text output.
3. Consider adding an explicit check in `SimpleConsoleSpanExporter.__init__()`:

```
if os.environ.get('NO_COLOR'):
    colors = 'never'
```

[LOW] — CSS Transition Has No prefers-reduced-motion Fallback

- **SDL Phase**: Implementation
- **WCAG Criterion**: 2.3.3 Animation from Interactions (Level AAA)
- **Category**: Documentation Animation / Motion
- **File**: `docs/extra/tweaks.css`, lines 63–64

Description

The login button hover state applies an opacity transition without a `prefers-reduced-motion` override:

```
.custom-login-button:hover {
  opacity: 0.7
}

/* Missing: no @media (prefers-reduced-motion: reduce) block */
```

Note: The `transition: opacity .25s` is declared on the base `.custom-login-button` rule (line 60). Users who have enabled "Reduce Motion" (macOS, Windows, iOS, Android) will still see this animation.

Impact
Users with vestibular disorders, migraine sensitivities, or other motion-triggered conditions who have configured their OS to reduce motion will still experience this transition. While this is a minor animation, best practice and WCAG AAA require all interaction-triggered animations to be suppressible.

Recommendation
Add the following to `docs/extra/tweaks.css`:

```
@media (prefers-reduced-motion: reduce) {
  .custom-login-button {
    transition: none;
  }
}
```

[LOW] — Rich Panel Box-Drawing Characters Render Verbatim in Screen-Reader Terminals

- **SDL Phase**: Implementation
- **WCAG Criterion**: 1.3.1 Info and Relationships (Level A — terminal accessibility best practice)
- **Category**: CLI Terminal Output / Rich UI
- **File**: `logfire/_internal/cli/run.py`, lines 282–293

Description
`print_otel_summary()` renders a Rich `Panel` using the `ROUNDED` box style:

```
panel = Panel(
    content,
```

```
    title='[bold blue]Logfire Summary[/bold blue]',
    border_style='blue',
    box=ROUNDED,
    padding=(1, 2),
)
console.print('\n')
console.print(panel)
```

The ROUNDED box style produces characters such as `╭`, `─`, `╮`, `│`, `╰`, `╯`. In screen-reader-enabled terminal emulators (Windows Narrator + CMD, BRLTTY + Linux VTE, Orca + GNOME Terminal), these box-drawing characters may be:

- Announced individually as their Unicode names ("BOX DRAWINGS LIGHT ARC DOWN AND RIGHT", etc.)
- Read as a rapid sequence of punctuation, creating an unusable noise burst before the actual content

The panel is rendered on `console.print(panel)` without checking `console.is_terminal`. When the output is a TTY, Rich does render the panel; when piped, Rich's auto-detection strips formatting, but screen-reader users often use TTY-mode terminals where the panel is fully rendered.

Impact

Users relying on screen readers in terminal emulators will hear box-drawing characters announced before reaching the summary content, creating a poor experience when running `logfire run` or `logfire inspect`.

Recommendation

Check for terminal vs. non-terminal output and provide a plain-text alternative:

```
if console.is_terminal:
    console.print(panel)
else:
    # Plain text fallback
    console.print('\n=== Logfire Summary ===')
    console.print(content)
    console.print('=' * 22)
```

Alternatively, consider providing a `--no-summary` flag (which already exists as `--no-summary` via `BooleanOptionalAction`) as the recommended mode for users who prefer plain output, and document this in the accessibility section of the CLI reference.

[LOW] — Silent Browser-Open Failure Provides No Distinct Error Message

- **SDL Phase**: Implementation
- **WCAG Criterion**: 3.3.1 Error Identification (Level A)
- **Category**: CLI Error Handling
- **File**: `logfire/_internal/cli/auth.py`, lines 63–66

Description

The `logfire auth` flow silently swallows browser-open failures:

```
try:
    webbrowser.open(frontend_auth_url, new=2)
except webbrowser.Error:
    pass  # <-- silent failure
sys.stderr.writelines(
    (
        f"Please open {frontend_auth_url} in your browser to authenticate if it hasn't already.\n",
        'Waiting for you to authenticate with Logfire...\n',
    )
)
```

The fallback message "if it hasn't already" is the only hint that the browser may not have opened. For users running text-based browsers, headless systems, or screen-reader-driven environments where `webbrowser.open()` may fail, there is no explicit error message distinguishing "browser opened" from "browser failed to open".

Impact

Users in headless environments, WSL without a GUI, or system configurations where `webbrowser` fails will not know to manually copy and open the URL. They may wait indefinitely for authentication to complete.

Recommendation

Distinguish the two states with distinct messages:

```
browser_opened = False
try:
    webbrowser.open(frontend_auth_url, new=2)
    browser_opened = True
except webbrowser.Error:
    pass

if browser_opened:
    sys.stderr.write(f'Browser opened. If it did not open, visit:\n  {frontend_auth_url}\n')
else:
    sys.stderr.write(f'Could not open browser automatically. Please open this URL manually:\n  {frontend_auth_url}\n')
sys.stderr.write('Waiting for you to authenticate with Logfire...\n')
```

[LOW] — Auth Error Messages Do Not Specify Which Command Variant to Run

- **SDL Phase**: Implementation
- **WCAG Criterion**: 3.3.3 Error Suggestion (Level AA)
- **Category**: CLI Error Messages
- **File**: `logfire/_internal/auth.py`, lines 146–164

Description

Several error messages in `UserTokenCollection.get_token()` suggest running `logfire auth` without specifying which region or URL is applicable:

```
raise LogfireConfigError(
    f'No user token was found matching the {base_url} Logfire URL. '
    'Please run `logfire auth` to authenticate.'
)
```

When a user has credentials for one region (e.g., EU) but is trying to use a different endpoint (e.g., US), `logfire auth` alone will attempt to authenticate against the default region and may not resolve the problem.

Impact
Users who follow the error message guidance will run `logfire auth` and may authenticate against a different endpoint than the one they need, wasting time and causing confusion. CLI users who rely on screen readers or are less familiar with the Logfire region system are especially vulnerable to this confusing feedback loop.

Recommendation
Improve error messages to specify the exact command needed:

```
raise LogfireConfigError(
    f'No user token found for {base_url}. '
    f'Run `logfire --base-url {base_url} auth` to authenticate for this endpoint.'
)
```

[LOW] — Search Icon Label Has No Accessible Text Alternative

- **SDL Phase**: Implementation
- **WCAG Criterion**: 1.1.1 Non-text Content (Level A), 4.1.2 Name, Role, Value (Level A)
- **Category**: Documentation Search Accessibility
- **File**: `docs/overrides/partials/search.html`, lines 6–11

Description
The search toggle label contains only SVG icons with no accessible name:

```
<label class="md-search__icon md-icon" for="__search">
  <svg ...><!-- search magnifying glass icon --></svg>
```

```
  <svg ...><!-- back arrow icon --></svg>
</label>
```

Neither SVG has `aria-hidden="true"`, `aria-label`, nor a `<title>` element. The `<label>` itself has no visible or accessible text. The associated `for="__search"` input is a hidden checkbox used as a CSS toggle, which means the label effectively acts as a toggle button. Screen readers may announce this as an unlabeled clickable element.

Impact

Screen reader users will not know what clicking this control does. VoiceOver may announce "image" for each SVG, while NVDA may skip unlabeled SVG icons entirely. The control's purpose (to open/close the search dialog) is not communicated.

Recommendation

Add `aria-hidden="true"` to both SVGs and an `aria-label` to the label element, or add a visually hidden span:

```
<label class="md-search__icon md-icon" for="__search" aria-label="Search">
  <svg aria-hidden="true" ...>...</svg>
  <svg aria-hidden="true" ...>...</svg>
</label>
```

Color Contrast Summary

Pair	Context	Ratio	AA Normal Text	AA Large/UI
`#E520E9` on `#FFFFFF`	Light mode links/nav	3.67:1	❌ FAIL	☑ PASS
`#E520E9` on `#1A0312`	Dark mode links/nav	5.32:1	☑ PASS	☑ PASS
`#E520E9` on `#321A2C`	Header/nav bar	3.88:1	❌ FAIL	☑ PASS
`#FFFFFF` on `#321A2C`	Header text	13.83:1	☑ PASS AAA	☑ PASS AAA

Pair	Context	Ratio	AA Normal Text	AA Large/UI
`#FFFFFF` on `#1A0312`	Dark body text	18.09:1	☑ PASS AAA	☑ PASS AAA
Console green (timestamp)	Rich `green` on default terminal	N/A — cosmetic	decorative only	—
Console red (errors)	Rich `red` on default terminal	Varies	not sole indicator after fix	—

Remediation Priority Matrix

Priority	Finding	Effort	Impact	WCAG Level
P0	Log level conveyed by color only	Low (1 file)	Critical	A
P1	Primary color contrast failure (light mode)	Low (CSS only)	High	AA
P1	Search results list role="presentation"	Low (1 word)	High	A
P2	Unicode-only status indicators	Low (1 file)	Medium	A
P2	Search result links tabindex="-1"	Medium (JS)	Medium	A
P2	Screenshot alt text	Medium (many files)	Medium	A
P2	NO_COLOR not documented	Low (docs)	Medium	—
P3	prefers-reduced-motion CSS	Low (2 lines)	Low	AAA
P3	Rich panel box characters	Medium (logic)	Low	A
P3	Silent browser failure	Low (auth.py)	Low	A

Priority	Finding	Effort	Impact	WCAG Level
P3	Auth error message specificity	Low (1 file)	Low	AA
P3	Search icon label	Low (HTML)	Low	A

Notes on Applicability

- **Python API** — Not applicable. All Python API surface (spans, metrics, logging methods, instrumentation wrappers) is developer-facing code with no interactive UI elements.
- **OTLP exporter** — Not applicable. Machine-to-machine binary protocol.
- **Test utilities** (`logfire.testing`) — Not applicable. Developer testing tools.
- **MkDocs documentation** — Partially applicable. The hosted documentation site at `logfire.pydantic.dev/docs` is a public-facing web interface covered by WCAG standards. The `docs/` source files are Markdown; the rendered HTML produced by Material for MkDocs is what users interact with. Findings in this category focus on customizations to the default Material theme (CSS overrides, HTML template overrides, JavaScript additions) since the base Material theme has its own accessibility considerations outside this codebase's control.

Report generated by Accessibility Audit Agent — SDL-aligned, WCAG 2.1/2.2 reference.